

Examen Práctico Convocatoria Extraordinaria (2024-2025)

- Dirección IP:
- Hora de entrega:
- Apellidos, nombre (en mayúsculas):

Instrucciones

1. Conecte el portátil a la WIFI Eduroam. Escriba la IP (debe empezar por 138) (Se puede utilizar: <https://www.cual-es-mi-ip.net>)
2. Baje el código de apoyo de Moodle y el enunciado.
3. El código debe compilar en Linux con las opciones -Wall -Wextra -Werror -ansi
4. La entrega es en Deliverit. Se puede entregar varias veces, y la última entrega será la que se corrija. Es obligatorio que la entrega compile.
5. El examen dura 90'. Escriba la hora de la última entrega en este enunciado y entréguelo.
6. Durante el examen se puede utilizar apuntes y está permitido el uso de man de GNU-Linux. Se considerará fraude académico la utilización de buscadores, ChatGPT o similares y compartir información con otros alumnos.

Parte lenguaje C

Se va a considerar el formato de imagen .pgm (un formato sencillo sin compresión) en su variante de texto, en la que todo el fichero es formato texto. (Nota: hay otra versión, más frecuente, pero que no se va a tratar, que tiene los valores de los píxeles en binario).

Concretamente, un fichero pgm en este formato consiste en:

- Una línea de texto con un identificador de la versión texto P2.
- Una línea de texto con el tamaño anchura y el tamaño altura separados por un espacio.
- Una línea con el valor máximo que puede haber (p.ej., un valor típico de valor máximo es 255 en imágenes de 1 byte por píxel y con rango de 0 a 255).
- Los valores de intensidad de los píxeles en caracteres ASCII separados por espacio(s), incluyendo saltos de línea.

En estos ejercicios, los valores de los píxeles serán enteros dentro del rango 0 a 255 (en formato texto, en esta variante P2), y pueden estar todos en una misma línea o en varias.

Por ejemplo, el contenido de un fichero .pgm de una 'mini' imagen de tamaño 8 (anchura) x 4 (altura) en esta versión de texto P2 podría ser :

```
P2
8 4
255
200 201 202 203 204 205 206 207
200 201 202 203 204 205 206 207
200 201 202 203 204 205 206 207
200 201 202 203 204 205 206 207
```

Se puede asumir que, fuera del identificador (string P2), el resto se puede representar mediante el tipo unsigned int (o dato_t, ver pgm.h). Los valores de los píxeles se representan mediante el tipo dato_t.

En este ejemplo los valores de los píxeles se han dividido en líneas para su mejor visualización, aunque no necesariamente es así: podrían estar en menos líneas, o incluso en una línea. La parte de la cabecera previa a los datos sí se compone de varias líneas.

Nota: se menciona que el estándar es menos estricto que lo indicado previamente, existiendo p.ej. la posibilidad de introducir comentarios en la cabecera. Estas posibilidades no se van a tratar.

Ejercicio 1. En el fichero `ej1.c` en lenguaje C que se deberá entregar, implemente una función `comprobar_pgm_P2` (ver prototipo en `pgm.h`, en ficheros de apoyo) que realice ciertas comprobaciones acerca del fichero cuyo nombre se proporciona en su único argumento:

- Si el fichero no existe o no es legible, se deberá acabar la ejecución devolviendo al sistema operativo el número 66, y escribir en salida error la siguiente línea (14 caracteres incluyendo el salto de línea):
`ERROR ENTRADA`
- Si a) el identificador de la variante no es P2, b) el número de valores de los píxeles existente después de la cabecera no es igual al número de píxeles (o tamaño, es decir, anchura x altura) de la imagen, o bien c) alguno de los valores de los píxeles supera el valor máximo, entonces se deberá cerrar el fichero abierto y acabar la ejecución devolviendo al sistema operativo el número 65, escribiendo así mismo en salida error la siguiente línea (14 caracteres incluyendo el salto de línea):
`ERROR FORMATO`

Si lo precisa (no es necesario, según se haga la lectura), se puede suponer que las líneas tienen una longitud máxima de `TAMANNO_LINEA_MAX` (ver `pgm.h`) caracteres incluyendo el posible salto de línea.

Puede considerar el fichero `ej1_main.c` (ver ficheros de apoyo) como simple ejemplo de llamada a la función `comprobar_pgm_P2`. Se pueden hacer algunas pruebas con el ejecutable generado a partir de ambos `ej1_main.c` y `ej1.c` empleando algunos ficheros de imágenes `pgm` válidas y no válidas que se encuentran en el directorio `apoyo/pgm` de los ficheros de apoyo (las no válidas aparecen así indicadas en el nombre del fichero). Y naturalmente, se deberían hacer pruebas adicionales con otras posibles situaciones de error.

FICHERO A ENTREGAR: `ej1.c`

PUNTOS: 2.5 puntos

Ejercicio 2. En un fichero `ej2.c` en lenguaje C, implemente una función `crear_pgm_P2` que reciba de entrada estándar la información y datos para generar una imagen `pgm P2` válida en salida estándar (ver prototipo en `pgm.h`).

Considere el fichero ejemplo `ej2_main.c` (ver ficheros de apoyo) que llama a `crear_pgm_P2` para crear el ejecutable correspondiente y hacer pruebas. Si `a.out` es el ejecutable generado por `ej2_main.c` y `ej2.c`, las siguientes llamadas:

```
echo 'P2
3 2
255
201 202 203 204 205 206' | ./a.out > imagen.pgm
```

```
echo 'P2
3 2
255
201 202 203
204 205 206' | ./a.out > imagen.pgm
```

deberán crear un fichero válido `imagen.pgm` correspondiente a una imagen de anchura 3 y altura 2, con valor máximo 255, y con los valores de los píxeles indicados (201, 202, 203, 204, 205 y 206).

Los valores de los píxeles deben mostrarse con el formato "%4d" (que toma 4 caracteres, con relleno si es necesario).

La imagen creada debe tener los valores de los píxeles en filas (altura) y columnas (anchura). Los valores consecutivos en las filas deben estar separados por un espacio, y después del último valor de cada fila deberá haber sólo un salto de línea.

Se puede suponer que el identificador es P2. La función a realizar deberá hacer las comprobaciones, y resolver las situaciones, que se indican a continuación:

- Si algún valor de los píxeles supera el valor máximo, se deberá considerar que se ha recibido dicho valor máximo (es decir, se fuerza el límite a dicho valor).
- Si se proporcionan menos valores de píxeles que el número de píxeles o tamaño de la imagen, se rellenarán los valores que faltan con 0.
- Si se proporcionan más valores de píxeles que el número de píxeles o tamaño de la imagen, se descartarán los valores de más.

FICHERO A ENTREGAR: ej2.c

PUNTOS: 2.5 puntos

Ejercicio 3. En un fichero ej3.c en lenguaje C, implemente:

- Una función `crear_imagen_rango` que reciba como argumentos la información de una cabecera pgm y que deberá devolver como resultado un puntero a `imagen_t` que deberá crearse y ser rellenada adecuadamente (ver detalles de la estructura en `pgm.h`).
La parte de los valores de los píxeles deberá ser un array asignado dinámicamente con los valores rellenos de un rango o intervalo $[0, (\text{anchura} * \text{altura}) - 1]$, es decir, el valor del primer píxel es 0, el del segundo es 1, y así sucesivamente, hasta el último valor que es igual a $(\text{anchura} * \text{altura}) - 1$.
- Una función `liberar_datos` que libere la memoria dinámica de la parte `datos_t` de la imagen apuntada por el argumento de entrada `im`, si `im` no es NULL. En otro caso, no se deberá hacer nada.

En la asignación de memoria dinámica deberán comprobar si hubiera algún error al respecto, en cuyo caso se deberá acabar la ejecución devolviendo al sistema operativo el número status 71, y escribir en salida error (6 caracteres incluyendo el salto de línea):

ERROR

En el fichero ilustrativo `ej3_main.c` (ver ficheros de apoyo) se muestra un `main` con llamadas a las funciones `obtener_datos` y `liberar_datos`. Se incluye así mismo un fichero auxiliar `mostrar_datos.c` con una función para mostrar los valores de los píxeles por si fuera de utilidad.

FICHERO A ENTREGAR: ej3.c

PUNTOS: 3 puntos

Parte lenguaje Bash

Ejercicio 4. En un fichero `ej4.sh` en lenguaje Bash, implemente un script que compruebe si los argumentos que se le pasan en línea de comandos corresponden a ficheros o directorios del directorio donde se está ejecutando.

Por cada argumento:

- Si se trata de un fichero se debe mostrar en salida estándar una línea que consista en el prefijo de 4 caracteres

(F):

seguido por el argumento en cuestión.

- Si se trata un directorio, análogamente al caso anterior aunque empleando en este caso el prefijo de 4 caracteres

(D):

- En otro caso, no se debe hacer nada.

Si no se le pasan argumentos, el script debe mostrar en salida error la línea

ERROR

y acabar la ejecución con exit status de 64.

Si se le pasan argumentos, debe acabar con un exit status que sea igual al número de argumentos que sean directorio.

Nota: por si fuera de utilidad, se muestra seguidamente un ejemplo en Bash referente a incrementar el valor de una variable N que representa un número:

```
N=0
echo $N
N=$((N+1))
echo $N
```

FICHERO A ENTREGAR: ej4.sh

PUNTOS: 2 puntos

Soluciones

Ejercicio 1.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "pgm.h"

void tratar_error_formato(FILE *);

void tratar_error_formato(FILE *fp) {
    fclose(fp);
    fprintf(stderr, "ERROR FORMATO\n");
    exit(65);
}

int comprobar_pgm_P2(char *nfichero) {
    FILE *fp;
    char identificador[3];
    dato_t anchura, altura;
    dato_t valor_maximo, valor;
    size_t cont_valores = 0;
    int res_fscanf;

    if ((fp = fopen(nfichero, "r")) == NULL) {
        fprintf(stderr, "ERROR ENTRADA\n");
        exit(66);
    }

    fscanf(fp, "%s", identificador);
    if (strcmp(identificador, "P2") != 0) tratar_error_formato(fp);

    res_fscanf = fscanf(fp, "%d %d\n%d\n", &anchura, &altura, &valor_maximo);

    while ((res_fscanf = fscanf(fp, "%d", &valor)) == 1) {
        if (valor > valor_maximo) tratar_error_formato(fp);
        cont_valores += 1;
    }
    if (cont_valores != (size_t) (anchura * altura)) tratar_error_formato(fp);

    fclose(fp);
    return 0;
}
```

Ejercicio 2.

```
#include <stdio.h>
#include "pgm.h"

void crear_pgm_P2() {
    int res_fscanf;
    char identificador[3];
    dato_t anchura, altura;
```

```

dato_t valor_maximo, valor_pixel;
size_t cont_valores = 0, cont_columna = 0;

fscanf(stdin, "%s", identificador); fprintf(stdout, "%s\n", identificador);
fscanf(stdin, "%d", &anchura); fprintf(stdout, "%d ", anchura);
fscanf(stdin, "%d", &altura); fprintf(stdout, "%d\n", altura);
fscanf(stdin, "%d", &valor_maximo); fprintf(stdout, "%d\n", valor_maximo);
while ((res_fscanf = fscanf(stdin, "%d", &valor_pixel)) == 1) {
    if (valor_pixel > valor_maximo) valor_pixel = valor_maximo;
    fprintf(stdout, "%4d", valor_pixel);
    cont_valores++;
    cont_columna++;
    if (cont_columna == anchura) {
        fprintf(stdout, "\n");
        cont_columna = 0;
    }
    else fprintf(stdout, " ");
    /* caso de exceso de valores */
    if (cont_valores == (size_t) (anchura * altura)) break;
}
/* caso de falta de valores */
while (cont_valores < (size_t) (anchura * altura)) {
    fprintf(stdout, "%4d", (dato_t) 0);
    cont_valores++;
    cont_columna++;
    if (cont_columna == anchura) {
        fprintf(stdout, "\n");
        cont_columna = 0;
    }
    else fprintf(stdout, " ");
}
return;
}

```

Ejercicio 3.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "pgm.h"

```

```

void tratar_error_mem_din(void);

```

```

void tratar_error_mem_din(void) {
    fprintf(stderr, "ERROR\n");
    exit(71);
}

```

```

imagen_t *crear_imagen_rango(char *identificador, dato_t anchura, dato_t altura, dato_t valor_maximo) {
    imagen_t *im;
    size_t cont;

    im = malloc(sizeof(imagen_t));
}

```

```

    if (im == NULL) tratar_error_mem_din();
    strcpy(im->identificador, identificador);
    im->anchura = anchura;
    im->altura = altura;
    im->valor_maximo = valor_maximo;
    im->datos = malloc(anchura * altura * sizeof(dato_t));
    if (im->datos == NULL) tratar_error_mem_din();
    for (cont = 0; cont < (size_t) im->anchura * im->altura; cont++)
        im->datos[cont] = cont;
    return (im);
}

void liberar_datos(imagen_t *im) {
    if (im != NULL) {
        free(im->datos);
    }
    return;
}

```

Ejercicio 4.

```

#!/usr/bin/bash

if test $# -eq 0; then
    echo 'ERROR' >&2
    exit 64
fi

CONT_DIR=0
for ele in "$@"; do
    test -f "$ele" && echo "(F):$ele"
    if test -d "$ele"; then
        echo "(D):$ele"
        CONT_DIR=$((CONT_DIR+1))
    fi
done

exit $CONT_DIR

```